

EfficientViM: Efficient Vision Mamba with Hidden State Mixer based State Space Duality

Sanghyeok Lee¹ Joonmyung Choi¹ Hyunwoo J. Kim^{2*}

¹Korea University ²KAIST

{cat0626, pizard}@korea.ac.kr hyunwoojkim@kaist.ac.kr

Abstract

For the deployment of neural networks in resource-constrained environments, prior works have built lightweight architectures with convolution and attention for capturing local and global dependencies, respectively. Recently, the state space model (SSM) has emerged as an effective operation for global interaction with its favorable linear computational cost in the number of tokens. To harness the efficacy of SSM, we introduce Efficient Vision Mamba (**EfficientViM**), a novel architecture built on hidden state mixer-based state space duality (**HSM-SSD**) that efficiently captures global dependencies with further reduced computational cost. With the observation that the runtime of the SSD layer is driven by the linear projections on the input sequences, we redesign the original SSD layer to perform the channel mixing operation within compressed hidden states in the HSM-SSD layer. Additionally, we propose multi-stage hidden state fusion to reinforce the representation power of hidden states and provide the design to alleviate the bottleneck caused by the memory-bound operations. As a result, the EfficientViM family achieves a new state-of-the-art speed-accuracy trade-off on ImageNet-1k, offering up to a 0.7% performance improvement over the second-best model SHViT with faster speed. Further, we observe significant improvements in throughput and accuracy compared to prior works, when scaling images or employing distillation training. Code is available at <https://github.com/mlvlab/EfficientViM>.

1. Introduction

Efficient vision architectures for resource-constrained environments have been consistently explored in computer vision tasks, including image classification [5, 19, 22, 38, 39, 52, 74], object detection [28, 44, 53, 61], segmenta-

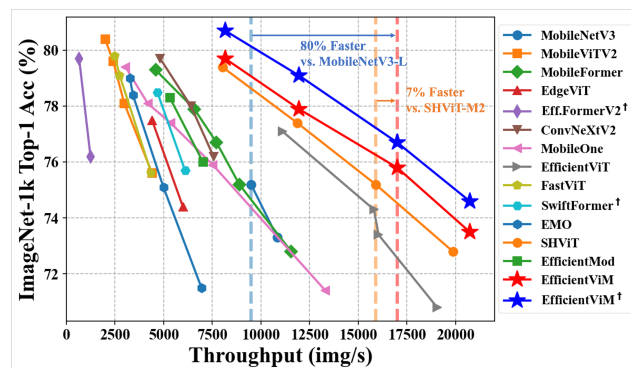


Figure 1. **Comparison of efficient networks on ImageNet-1K [9] classification.** The family of our EfficientViM, marked as red and blue stars, shows the best speed-accuracy trade-offs. † indicates the model trained with distillation following [55].

tion [20, 65, 67, 68, 75], etc. Earlier works [5, 15, 36] have explored the efficient convolutional neural networks (CNN). One such technique is depthwise separable convolution (DWConv), introduced in Xception [5], which has been widely adopted for modeling lightweight architectures, especially CNN-based networks including MobileNet [18, 19, 46] and following works [52, 59]. Meanwhile, with the advent of Vision Transformer (ViT) [11], attention mechanisms have become a key operation for capturing long-range dependencies within image patches. However, the quadratic cost of self-attention presents challenges for designing efficient architectures. To address this, prior works have attempted to approximate self-attention with reduced cost [6, 26, 62, 66], or restrict the number of tokens [1, 4, 7, 27, 35, 70]. For on-device deployment, several works [3, 33, 38, 39, 41, 58, 72, 73] devoted effort to developing hybrid ViTs combined with CNNs (DWConv). While prior works have demonstrated superior performance over traditional CNNs, the inherent quadratic complexity of self-attention in the number of tokens remains a major bottleneck limiting their efficiency and scalability. Recently, state space models (SSMs) [8, 12–14, 51] have emerged as a promising alternative to self-attention, of-

*Corresponding author.

fering a favorable linear computational complexity while maintaining a global receptive field. Mamba [8, 13] has introduced selective scanning mechanisms on SSM (S6) to process sequences with the hardware-aware parallel algorithm. Following Mamba, vision Mambas [21, 34, 69, 77] have extended the concept of SSM to vision tasks. These works have studied multi-path scanning mechanisms to address the causal constraints of SSM that are not desirable for image processing. More recent works like VSSD [49] and Linfusion [32] further eliminated the causal mask in the state space duality (SSD) of Mamba2 [8], introducing non-causal state space duality (NC-SSD). While vision Mambas demonstrate improved performance over previous SOTA methods, they still have relatively slower speeds than lightweight vision models. Further, we have observed that the major bottleneck of the previous SSD is the linear projection in gating operation and output projection.

In this paper, we present an **Efficient Vision Mamba (EfficientViM)**, a new family of mamba-based lightweight vision backbone built with a fast and effective SSD layer called **Hidden State Mixer-based SSD (HSM-SSD)**. In the HSM-SSD layer, we transfer the channel mixing operations of the standard SSD layer, including linear projection and gating function, from the image feature space to the hidden state space. These hidden states serve as compressed latent representations of the input. We observe that this design mitigates the major bottleneck of the SSD layer while maintaining the generalization ability of the model.

Also, we introduce a multi-stage hidden state fusion approach that generates the predictions by combining the original logits with those derived from the hidden states at each stage, leading to an enhanced representation power of hidden states. After breaking down the runtime of the HSM-SSD layer, we present the macro design that minimizes memory-bound operations, prioritizing practical performance in real-world applications over theoretical metrics like FLOPs. Through extensive experiments, we demonstrate that our EfficientViM achieves the new speed-accuracy state-of-the-art trade-off as shown in Figure 1. In particular, EfficientViM-M2 outperforms the previous SOTA model SHViT [72], and pioneering work ModelNetV3 [18] with a 0.6% performance improvement even bringing about 7% and 80% speed-ups, respectively. In summary, the contributions of EfficientViM are threefold:

- We propose a novel mamba-based lightweight architecture called EfficientViM, leveraging the linear computational cost of the global token mixer.
- We introduce HSM-SSD, which makes the major overhead of the SSD layer controllable by adjusting the number of hidden states.
- With a design minimizing memory-bound operations and incorporating multi-stage hidden state fusion, EfficientViM achieves the best speed-accuracy tradeoffs.

2. Preliminaries

State space models (SSM). Inspired by the linear time-invariant (LTI) continuous system, an SSM maps an input sequence $x(t) \in \mathbb{R}$ to an output sequence $y(t) \in \mathbb{R}$ as

$$h'(t) = \hat{\mathbf{A}}h(t) + \hat{\mathbf{B}}x(t), \quad y(t) = \hat{\mathbf{C}}h(t), \quad (1)$$

where $h(t) \in \mathbb{R}^{N \times 1}$ is a hidden state, $\hat{\mathbf{A}} \in \mathbb{R}^{N \times N}$, $\hat{\mathbf{B}} \in \mathbb{R}^{N \times 1}$, $\hat{\mathbf{C}} \in \mathbb{R}^{1 \times N}$ are the projection matrix, and N is the number of states. To adapt this continuous-time system for discrete data in deep learning, given the multivariate input sequences $\mathbf{x} = [\mathbf{x}_1^\top, \dots, \mathbf{x}_L^\top]^\top \in \mathbb{R}^{L \times D}$ with $\forall_t \mathbf{x}_t \in \mathbb{R}^{1 \times D}$, Mamba [13] first generates parameters as $\hat{\mathbf{B}}, \mathbf{C} = \mathbf{x}\mathbf{W}_B, \mathbf{x}\mathbf{W}_C \in \mathbb{R}^{L \times N}$, $\Delta = \mathbf{x}\mathbf{W}_\Delta \in \mathbb{R}^L$, where $\mathbf{W}_B, \mathbf{W}_C \in \mathbb{R}^{D \times N}$, $\mathbf{W}_\Delta \in \mathbb{R}^{D \times 1}$ are learnable matrices. Then, the discretized form of SSM with zero-order hold discretization is defined as

$$\mathbf{h}_t = \mathbf{A}_t\mathbf{h}_{t-1} + \mathbf{B}_t^\top \mathbf{x}_t, \quad \mathbf{y}_t = \mathbf{C}_t\mathbf{h}_t, \quad (2)$$

where $\mathbf{y} \in \mathbb{R}^{L \times D}$, $\mathbf{h}_t \in \mathbb{R}^{N \times D}$, $\mathbf{A}_t = e^{\Delta_t \hat{\mathbf{A}}} \in \mathbb{R}^{N \times N}$, $\mathbf{B}_t = (\Delta_t \hat{\mathbf{A}})^{-1}(e^{\Delta_t \hat{\mathbf{A}}} - \mathbf{I}) \cdot \Delta_t \hat{\mathbf{B}}_t \approx \Delta_t \hat{\mathbf{B}}_t \in \mathbb{R}^{1 \times N}$. In this formulation, $\hat{\mathbf{A}} \in \mathbb{R}^{N \times N}$ is a learnable diagonal matrix, and all projection matrices $\mathbf{A}_t, \mathbf{B}_t, \mathbf{C}_t$ enable the linear time-variant discrete system that selectively attends to inputs $\mathbf{x}$ and hidden state $\mathbf{h}$ of each timestamp t .

State space duality (SSD). Mamba2 [8] further simplifies the diagonal form of the evolution matrix $\hat{\mathbf{A}}$ into the scalar form as $\hat{a} \in \mathbb{R}$ resulting in $\mathbf{a} \in \mathbb{R}^L$ via the same discretization step. Then, the state space duality (SSD) reformulates Equation (2) as matrix transformation:

$$\mathbf{y} = \text{SSD}(\mathbf{x}, \mathbf{a}, \mathbf{B}, \mathbf{C}) = (\mathbf{M} \odot (\mathbf{C}\mathbf{B}^\top)) \mathbf{x},$$

$$\mathbf{M}_{ij} = \begin{cases} \prod_{k=j+1}^i \mathbf{a}_k & \text{if } i > j \\ 1 & \text{if } i = j \\ 0 & \text{if } i < j \end{cases}, \quad (3)$$

where $\mathbf{M} \in \mathbb{R}^{L \times L}$, and $\odot$ indicates Hadamard product. Note that the lower triangular matrix $\mathbf{M}$ acts as a causal mask, which is suboptimal for image processing. To address this, non-causal SSD (NC-SSD) [32, 77] has been studied as an alternative to SSD by defining the mask as $\mathbf{M}_{ij} = \prod_{k=j+1}^L \mathbf{a}_k$, resulting in $\mathbf{M}_{1j} = \mathbf{M}_{2j} = \dots = \mathbf{M}_{Lj}$. Further, in VSSD [77], it is simplified as $\mathbf{M}_{ij} = \mathbf{a}_j$ resulting in

$$\mathbf{y} = \text{NC-SSD}(\mathbf{x}, \mathbf{a}, \mathbf{B}, \mathbf{C}) = \mathbf{C}\mathbf{h},$$

$$\mathbf{h} = (\mathbf{a}\mathbb{1}_N^\top \odot \mathbf{B})^\top \mathbf{x} = \sum_{i=1}^L \mathbf{a}_i \mathbf{B}_i^\top \mathbf{x}_i, \quad (4)$$

where $\mathbb{1}_N \in \mathbb{R}^N$ is one vector for broadcasting $\mathbf{a}$. Since the cumulative multiplication of $\mathbf{a}$ restricts the receptive fields as discussed in [49, 50], we adopt this version of NC-SSD as our starting point for an efficient token mixer.

Complexity	Method
$\mathcal{O}(LD^2 + L^2D)$	Attention [60]
$\mathcal{O}(LD^2)$	Linear Attention [24]
$\mathcal{O}(LD^2 + LND)$	SSD [8], NC-SSD [49]
$\mathcal{O}(ND^2 + LND)$	HSM-SSD

Table 1. **Complexity comparison of global tokens mixers.** L : # tokens, N : # states, D : # channels.

3. Method

We present a hidden state mixer-based SSD (**HSM-SSD**) for capturing global context with reduced costs, detailed in Section 3.1. Then, we discuss the additional techniques to improve both speed and performance with HSM-SSD in Section 3.2. After that, we outline the overall architecture and block designs to construct **EfficientViM** in Section 3.3.

3.1. Hidden State Mixer-based SSD

We start with a brief discussion on the computational cost of the NC-SSD layer depicted in Figure 2a. The entire process of the NC-SSD layer can be summarized as

$$\hat{\mathbf{B}}, \mathbf{C}, \Delta, \mathbf{x}, \mathbf{z} = \text{Linear}(\mathbf{x}_{\text{in}}) \quad (5)$$

$$\mathbf{a}, \mathbf{B} = \text{Discretization}(\hat{\mathbf{a}}, \hat{\mathbf{B}}, \Delta), \quad (6)$$

$$\mathbf{B}, \mathbf{C}, \mathbf{x} := \text{DWConv}(\mathbf{B}, \mathbf{C}, \mathbf{x}), \quad (7)$$

$$\mathbf{y} = \text{NC-SSD}(\mathbf{x}, \mathbf{a}, \mathbf{B}, \mathbf{C}), \quad (8)$$

$$\mathbf{x}_{\text{out}} = \text{Linear}(\mathbf{y} \odot \sigma(\mathbf{z})), \quad (9)$$

where $\mathbf{x}_{\text{in}}, \mathbf{x}_{\text{out}}, \mathbf{x}, \mathbf{z} \in \mathbb{R}^{L \times D}$, and σ is the activation function. The computational costs of Equations (5) to (7) with a constant kernel size is $\mathcal{O}(LD^2 + LND)$. Subsequently, executing NC-SSD and the output projection requires $\mathcal{O}(LND)$ and $\mathcal{O}(LD^2)$, respectively. Given that the number of states N is typically much smaller than the number of channels D (i.e., $N \ll D$), the overall complexity is mainly driven by the linear projections involved in generating $\mathbf{x}$, $\mathbf{z}$, and $\mathbf{x}_{\text{out}}$, leading to $\mathcal{O}(LD^2)$. Therefore, optimizing the linear projection in SSD blocks is crucial for scalability.

We delve into optimizing these computations for efficient layer designs. NC-SSD (Equation (4)) can be factorized into two steps. First, it obtains shared global hidden state $\mathbf{h} \in \mathbb{R}^{N \times D}$ through a weighted linear combination of input states $\mathbf{B}_i^\top \mathbf{x}_i \in \mathbb{R}^{N \times D}$ using importance weights $\mathbf{a} \in \mathbb{R}^L$. Second, the outputs for each input are generated by projecting a hidden state with their corresponding $\mathbf{C} \in \mathbb{R}^{L \times N}$. Here, if we denote the projected input $\mathbf{x}$ as $\mathbf{x}_{\text{in}} \mathbf{W}_{\text{in}}$ removing DWConv, the following holds:

$$\begin{aligned} \mathbf{h} &= (\mathbf{a} \mathbb{1}_N^\top \odot \mathbf{B})^\top (\mathbf{x}_{\text{in}} \mathbf{W}_{\text{in}}) \\ &= ((\mathbf{a} \mathbb{1}_N^\top \odot \mathbf{B})^\top \mathbf{x}_{\text{in}}) \mathbf{W}_{\text{in}} = \mathbf{h}_{\text{in}} \mathbf{W}_{\text{in}}, \end{aligned} \quad (10)$$

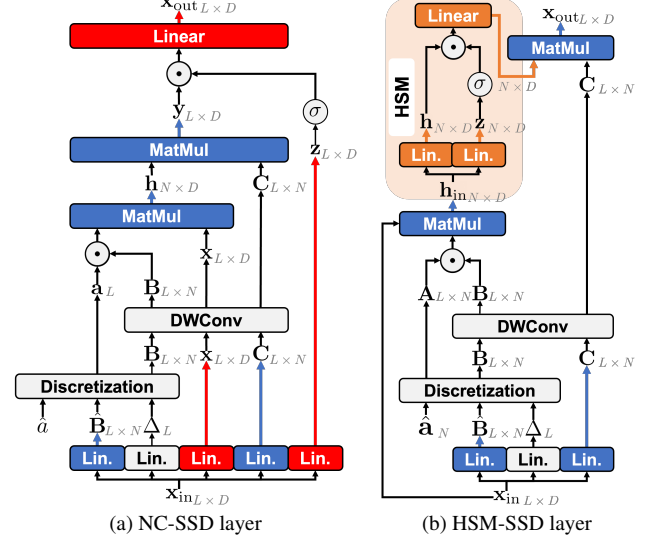


Figure 2. **Illustration of (left) NC-SSD and (right) HSM-SSD layer.** In the HSM-SSD layer, the computationally heavy projections are handled with the reduced hidden state in HSM as highlighted. Red, blue, and orange colors indicate the operation requiring the complexities of $\mathcal{O}(LD^2)$, $\mathcal{O}(LND)$, and $\mathcal{O}(ND^2)$.

where $\mathbf{W}_{\text{in}} \in \mathbb{R}^{D \times D}$, and $\mathbf{h}_{\text{in}} = (\mathbf{a} \mathbb{1}_N^\top \odot \mathbf{B})^\top \mathbf{x}_{\text{in}} \in \mathbb{R}^{N \times D}$. By computing $\mathbf{h}_{\text{in}}$ first, we perform a linear projection onto the hidden state space. This approach reduces the cost from $\mathcal{O}(LD^2)$ to $\mathcal{O}(ND^2)$ which relies on the number of states. In other words, we could alleviate the major costs of the layer by adjusting the number of states such that $N \ll L$.

Hidden State Mixer. The next step is to alleviate the cost of gating and output projection in Equation (9), which still remains $\mathcal{O}(LD^2)$. To address this, we focus on a shared global hidden state $\mathbf{h}$. Note that the hidden state $\mathbf{h}$ itself is the reduced latent array that compresses the input data with a significantly smaller sequence length N . Based on this observation, we propose a *Hidden State Mixer (HSM)* that performs channel mixing, including the gating and output projection, directly on the reduced latent array $\mathbf{h}$ as highlighted in Figure 2b. To this end, we approximate the output of the NC-SSD layer as follows:

$$\begin{aligned} \mathbf{x}_{\text{out}} &= f(\mathbf{y}) \\ &= \text{Linear}(\mathbf{y} \odot \sigma(\mathbf{z})) \\ &= (\mathbf{C} \mathbf{h} \odot \sigma(\mathbf{x}_{\text{in}} \mathbf{W}_{\text{z}})) \mathbf{W}_{\text{out}} \\ &\approx \mathbf{C} ((\mathbf{h} \odot \sigma(\mathbf{h}_{\text{in}} \mathbf{W}_{\text{z}})) \mathbf{W}_{\text{out}}) = \mathbf{C} f(\mathbf{h}), \end{aligned} \quad (11)$$

where $\mathbf{y} = \mathbf{C} \mathbf{h}$ from Equation (4), and f indicates channel mixing of gating function followed by linear projection with the learnable matrix $\mathbf{W}_{\text{z}}, \mathbf{W}_{\text{out}} \in \mathbb{R}^{D \times D}$. Contrary to the original NC-SSD layer where $\mathbf{C} \mathbf{h}$ is computed first and then fed to f , we apply the gating and projection directly to the hidden states with HSM. Then, the final output $\mathbf{x}_{\text{out}}$ is generated by projecting updated hidden states with $\mathbf{C}$. Consequently, the total complexity of capturing global

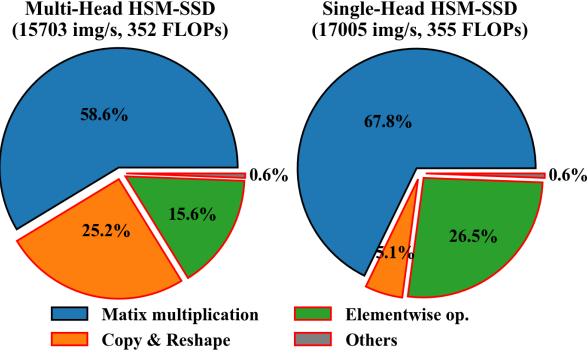


Figure 3. **Runtime breakdown of HSM-SSD with EfficientViM-M2.** The operations highlighted in red are memory-bound.

context in the HSM-SSD layer becomes $\mathcal{O}(ND^2 + LND)$, which is negligible as N gets smaller. Refer to Table 1 for a comparison of the big- $\mathcal{O}$ complexities with previous global token mixers.

Proposition 1. Let $N = L$, $\mathbf{a} \mathbb{1}_L^\top \odot \mathbf{B} = \mathbb{I}_L$, and $\mathbf{C} \in \mathbb{R}^{L \times L}$ be diagonal. Then, $\text{HSM-SSD}(\mathbf{x}, \mathbf{a}, \mathbf{B}, \mathbf{C})$ is equivalent to $\text{NC-SSD}(\mathbf{x}, \mathbf{a}, \mathbf{B}, \mathbf{C})$ including gating and output projection, as $\mathbf{x}_{\text{out}} = f(\mathbf{y}) = \mathbf{C}f(\mathbf{h})$. See supplement for proof.

Remark 1. Although we utilize gating and linear projection for the HSM to emulate the operation in the original SSD layer, other methods are also available. For example, inspired by the Perceiver architecture [23], we could implement a global token mixer for the function f in Eq. (11) to efficiently handle high-dimensional inputs using a reduced set of latent variables. Moreover, HSM-SSD can recursively apply HSM-SSD itself as the hidden state mixer, further reducing computational complexity and forming a recurrent architecture unrolled across depth.

3.2. HSM-SSD layer

Multi-stage hidden state fusion. To further improve the performance of EfficientViM, we introduce a *multi-stage hidden-state fusion (MSF)* mechanism that fuses the prediction logits leveraging hidden states from multiple stages of the network. Let $\{\mathbf{h}^{(s)}\}_{s=1}^S$ denote the hidden states at the last block of each stage s , where S is the total number of stages. For each $\mathbf{h}^{(s)}$, we compute a global representation $\hat{\mathbf{h}}^{(s)}$ through a simple average over the hidden states:

$$\hat{\mathbf{h}}^{(s)} = \frac{1}{N} \sum_{i=1}^N \mathbf{h}_i^{(s)}. \quad (12)$$

Then, each global representation $\hat{\mathbf{h}}^{(s)} \in \mathbb{R}^D$ is normalized and projected to generate its corresponding logits $\mathbf{z}^{(s)} \in \mathbb{R}^c$, where c indicates the number of classes. We set the final logit $\mathbf{z}$ of EfficientViM as a weighted sum of the logits from all stages, including the original logit $\mathbf{z}^{(0)}$ obtained

Algorithm 1 HSM-SSD Layer

Input: $\mathbf{x}_{\text{in}} \in \mathbb{R}^{L \times D}$

Output: $\mathbf{x}_{\text{out}} \in \mathbb{R}^{L \times D}$

- 1: $\hat{\mathbf{B}}, \mathbf{C}, \Delta \leftarrow \text{Linear}(\mathbf{x}_{\text{in}})$ $\triangleright \mathcal{O}(LND)$
- 2: $\hat{\mathbf{B}}, \mathbf{C} \leftarrow \text{DWConv}(\hat{\mathbf{B}}, \mathbf{C})$ $\triangleright \mathcal{O}(LNK^2D)$
- 3: $\mathbf{A}, \mathbf{B} \leftarrow \text{Discretization}(\hat{\mathbf{a}}, \hat{\mathbf{B}}, \Delta)$ $\triangleright \mathcal{O}(LD)$
- 4: $\mathbf{h}_{\text{in}} \leftarrow (\mathbf{A} \odot \mathbf{B})^\top \mathbf{x}_{\text{in}}$ $\triangleright \mathcal{O}(LND)$
- 5: $\mathbf{h}, \mathbf{z} \leftarrow \text{Linear}(\mathbf{h}_{\text{in}})$ $\triangleright \mathcal{O}(ND^2)$
- 6: $\mathbf{h} \leftarrow \text{Linear}(\mathbf{h} \odot \sigma(\mathbf{z}))$ $\triangleright \mathcal{O}(ND^2)$
- 7: $\mathbf{x}_{\text{out}} \leftarrow \mathbf{C}\mathbf{h}$ $\triangleright \mathcal{O}(LND)$
- 8: **Return** $\mathbf{x}_{\text{out}}$

from the output of the last stage, which is defined as

$$\mathbf{z} = \sum_{s=0}^S \hat{\beta}^{(s)} \mathbf{z}^{(s)}, \quad (13)$$

$$\hat{\beta}^{(s)} = \frac{\exp(\beta^{(s)})}{\sum_{i=0}^S \exp(\beta^{(i)})},$$

where $\beta^{(s)}$ is learnable scalar. By training with this combined logit, we explicitly reinforce the representational power of the hidden states, as they contribute to the final predictions. It also enriches the information by integrating both low-level and high-level features, thereby enhancing the generalization ability of the model during inference.

Single-head HSM-SSD. The multi-head design in the attention mechanism [60] allows it to selectively attend to the features from independent representation subspaces within each head. SSD-based models [8, 49] generally adopt a multi-head variant called multi-input SSD, where the input $\mathbf{x}$ and $\mathbf{a}$ are defined for each head while $\mathbf{B}$, and $\mathbf{C}$ are shared across the head. However, recent work [72] has pointed out that a significant portion of real runtime in multi-head self-attention is driven by memory-bound operations.

In our preliminary experiments, we also found that multi-head configuration has become a bottleneck for HSM-SSD as summarized in Figure 3. As shown in the figure, the real runtime of multi-head HSM-SSD is largely bounded by memory access, requiring almost a quarter of the total runtime. Hence, we eliminate all tensor manipulation caused by multi-head (e.g., reshape, copy operation). Concurrently, to mimic the capability of multi-head in capturing diverse relationships, we set $\Delta \in \mathbb{R}^{L \times N}$, $\hat{\mathbf{a}} \in \mathbb{R}^N$, enabling the importance weights $\mathbf{A} \in \mathbb{R}^{L \times N}$ to estimate the importance of the tokens per state. Then, the input for the hidden state mixer becomes

$$\mathbf{h}_{\text{in}} = (\mathbf{A} \odot \mathbf{B})^\top \mathbf{x}_{\text{in}}. \quad (14)$$

As a result, our single-head HSM-SSD with state-wise importance weights achieved higher throughput (17,005 img/s) with single-head compared to multi-head (15,703

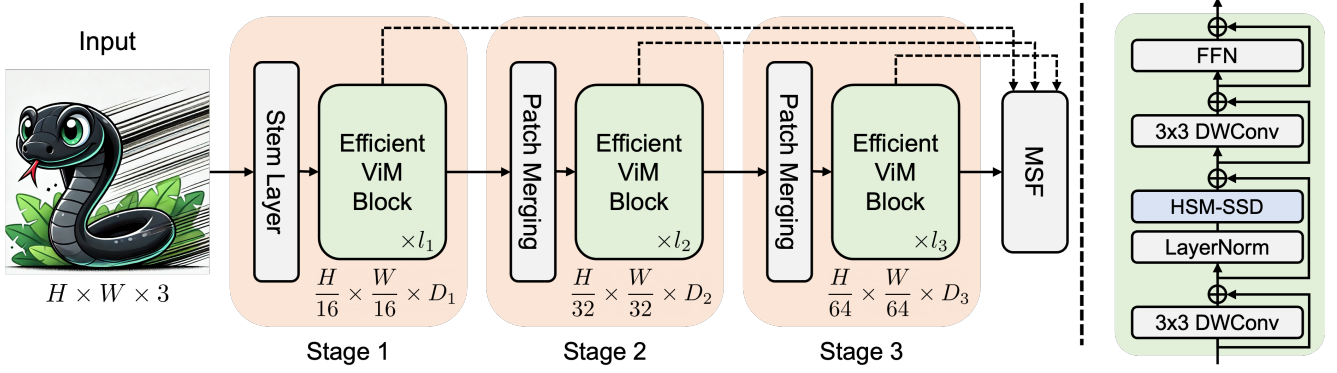


Figure 4. (left) Overall architecture and (right) block design of EfficientViM. The dotted line indicates a skip connection for multi-stage hidden state fusion (MSF). Illustration of the HSM-SSD layer in the EfficientViM block is presented in Figure 2.

img/s) along with competitive performance (see Table 9.b) under similar FLOPs. The pseudocode of the single-head HSM-SSD layer is provided in Algorithm 1.

3.3. EfficientViM

In this subsection, we present EfficientViM, an efficient vision mambas built upon the HSM-SSD layer. The overall architecture is illustrated in Figure 4.

Block design. In each block of EfficientViM, we sequentially stack the HSM-SSD layer and a feed-forward network (FFN) to facilitate global information aggregation and channel interaction, respectively. FFN layer consists of two consecutive 1×1 convolution layers, known as pointwise convolution, with an expansion ratio of 4. To capture the local context with minimal computational costs, we incorporate a 3×3 depthwise convolution (DWConv) layer before both the NC-SSD and FFN layers. Each layer is combined with a residual connection using a layer scale following [56, 57]. For normalization, we apply layer normalization (LN) only before the HSM-SSD layer for numerical stability, while batch normalization (BN) is used for DWConv and FFN considering its faster speed over LN.

Overall architecture. The stem layer first maps the $H \times W \times 3$ input image to the down-sized feature map $\frac{H}{16} \times \frac{W}{16} \times D_1$ through the four consecutive 3×3 convolutional layers with the stride of 2. Then, the resulting feature map is fed into the three stages built with EfficientViM blocks. To achieve hierarchical architecture and improve efficiency, we downscale the feature map while increasing the number of channels at the end of each stage via the down-sampling layer adopted from [33, 46, 77]. Regarding activation functions, we use SiLU only in the HSM-SSD layer, and others use ReLU [40] since the latencies of the complex activation functions (*e.g.*, Gelu [17], DynamicReLU [2], etc.) are largely dependent on devices, as discussed in previous works [33, 59, 72]. Detailed architecture specifications of EfficientViM variants are available in Table 2.

Model	# Blocks	# Channels	# States
EfficientViM-M1	[2, 2, 2]	[128, 192, 320]	[49, 25, 9]
EfficientViM-M2	[2, 2, 2]	[128, 256, 512]	[49, 25, 9]
EfficientViM-M3	[2, 2, 2]	[224, 320, 512]	[49, 25, 9]
EfficientViM-M4	[3, 4, 2]	[224, 320, 512]	[64, 32, 16]

Table 2. Specification of EfficientViM variants.

4. Experiments

In this section, we first demonstrate the effectiveness of EfficientViM on image classification (Section 4.1). Then, we conduct experiments to analyze the extensibility of EfficientViM on dense predictions (Section 4.2). We also provide ablation studies and analysis of EfficientViM (Section 4.3). See the supplement for implementation details and more experiments.

4.1. Image Classification.

Comparison with efficient vision backbones. For comparison of EfficientViM with prior works, we conduct ImageNet-1K [9] classification. To validate the effectiveness of EfficientViM in speed-accuracy trade-offs, we measure the throughput (im/s), and latency (ms) with the batch size of 256 on an NVIDIA RTX 3090 along with the accuracy, and present the results in Table 3. EfficientViM outperforms all previous efficient networks in both speed and accuracy. After training 450 epochs, EfficientViM-M1 shows a competitive performance with MobileNetV3-L 0.75 [18] and EfficientViT-M3 [33] while achieving about 90% and 30% speedup. Further, EfficientViM-M2 achieves about $4 \times$ faster speed, with a 0.2% performance improvement compared to MobileViTV2 0.75 [39] and FastViT-T8 [58]. EfficientViM-M3 and M4 achieve 77.9% and 79.7% accuracy, respectively, surpassing all previous works in throughput and accuracy within each section. Also, EfficientViM consistently outperforms the previous SOTA network, SHViT [72], across model sizes while reducing latency, which demonstrates the superiority of EfficientViM.

Method	Venue	Input Size	Epochs	Token Mixer	Throughput (im/s) $\uparrow$ Thr _{rel} $\uparrow$		Latency (ms) $\downarrow$	Top-1 (%) $\uparrow$	Params (M)	FLOPs (M)
MobileViTV2 0.5 [39]	Arxiv 2022	256 ²	300	Att.	6,702	$\times 0.32$	0.149	70.2	1.4	466
MobileOne-S0 [59]	CVPR 2023	224 ²	300	Conv	13,313	$\times 0.64$	0.075	71.4	2.1	275
EMO-1M [73]	ICCV 2023	224 ²	300	Att.	6,945	$\times 0.34$	0.144	71.5	1.3	261
MobileFormer-96M [3]	CVPR 2022	224 ²	450	Att.	11,554	$\times 0.56$	0.087	72.8	4.6	96
SHViT-S1 [72]	CVPR 2024	224 ²	300	Att.	19,868	$\times 0.96$	0.050	72.8	6.3	241
EfficientViM-M1	-	224 ²	300	SSD	20,731	$\times 1.00$	0.048	72.9	6.7	239
MobileNetV3-L 0.75 [18]	ICCV 2019	224 ²	600	Conv	10,846	$\times 0.52$	0.092	73.3	4.0	155
EfficientViT-M3 [33]	CVPR 2023	224 ²	300	Att.	16,045	$\times 0.77$	0.062	73.4	6.9	263
EfficientViM-M1	-	224 ²	450	SSD	20,731	$\times 1.00$	0.048	73.5	6.7	239
EfficientFormerV2-S0 [30]	NeurIPS 2022	224 ²	300	Att.	1,350	$\times 0.08$	0.741	73.7	3.5	407
EfficientViT-M4 [33]	CVPR 2023	224 ²	300	Att.	15,807	$\times 0.93$	0.063	74.3	8.8	299
EdgeViT-XXS [41]	ECCV 2022	224 ²	300	Att.	5,990	$\times 0.35$	0.167	74.4	4.1	556
EMO-2M [73]	ICCV 2023	224 ²	300	Att.	4,990	$\times 0.29$	0.200	75.1	2.3	439
MobileNetV3-L 1.0 [18]	ICCV 2019	224 ²	600	Conv	9,493	$\times 0.56$	0.105	75.2	5.4	217
MobileFormer-151M [3]	CVPR 2022	224 ²	450	Att.	8,890	$\times 0.52$	0.112	75.2	7.6	151
SHViT-S2 [72]	CVPR 2024	224 ²	300	Att.	15,899	$\times 0.93$	0.063	75.2	11.4	366
EfficientViM-M2	-	224 ²	300	SSD	17,005	$\times 1.00$	0.059	75.4	13.9	355
MobileViTV2 0.75 [39]	Arxiv 2022	256 ²	300	Att.	4,409	$\times 0.26$	0.227	75.6	2.9	1030
FastViT-T8 [58]	ICCV 2023	256 ²	300	Att.	4,365	$\times 0.26$	0.229	75.6	3.6	705
EfficientViM-M2	-	224 ²	450	SSD	17,005	$\times 1.00$	0.059	75.8	13.9	355
EfficientMod-XXS [37]	ICLR 2024	224 ²	300	Att.	7022	$\times 0.59$	0.142	76.0	4.7	583
ConvNeXtV2-A [64]	CVPR 2023	224 ²	300	Conv	7563	$\times 0.63$	0.132	76.2	3.7	552
EfficientViT-M5 [33]	CVPR 2023	224 ²	300	Att.	11,105	$\times 0.93$	0.090	77.1	12.4	522
MobileOne-S2 [59]	CVPR 2023	224 ²	300	Conv	5,360	$\times 0.45$	0.187	77.4	7.8	1299
SHViT-S3 [72]	CVPR 2024	224 ²	300	Att.	11,873	$\times 0.99$	0.084	77.4	14.2	601
EdgeViT-XS [41]	ECCV 2022	224 ²	300	Att.	4,405	$\times 0.37$	0.227	77.5	6.7	1136
EfficientViM-M3	-	224 ²	300	SSD	11,952	$\times 1.00$	0.084	77.6	16.6	656
MobileFormer-294M [3]	CVPR 2022	224 ²	450	Att.	6,576	$\times 0.55$	0.152	77.9	11.4	294
EfficientFormerV2-S1 [30]	NeurIPS 2022	224 ²	300	Att.	1,248	$\times 0.10$	0.801	77.9	6.1	668
EfficientViM-M3	-	224 ²	450	SSD	11,952	$\times 1.00$	0.084	77.9	16.6	656
ConvNeXtV2-F [64]	CVPR 2023	224 ²	300	Conv	6,405	$\times 0.78$	0.156	78.0	5.2	785
MobileViTV2 1.0 [39]	Arxiv 2022	256 ²	300	Att.	2,977	$\times 0.36$	0.336	78.1	4.9	1844
MobileOne-S3 [59]	CVPR 2023	224 ²	300	Conv	4,181	$\times 0.51$	0.239	78.1	10.1	1896
EfficientMod-XS [37]	ICLR 2024	224 ²	300	Att.	5,321	$\times 0.65$	0.188	78.3	6.6	778
EMO-6M [73]	ICCV 2023	224 ²	300	Att.	3,266	$\times 0.40$	0.306	79.0	6.1	961
FastViT-T12 [58]	ICCV 2023	256 ²	300	Att.	2,741	$\times 0.34$	0.365	79.1	6.8	1419
MobileFormer-508M [3]	CVPR 2022	224 ²	450	Att.	4,586	$\times 0.56$	0.218	79.3	14.0	508
MobileOne-S4 [59]	CVPR 2023	224 ²	300	Conv	3,041	$\times 0.37$	0.329	79.4	14.8	2978
SHViT-S4 [72]	CVPR 2024	256 ²	300	Att.	8,024	$\times 0.98$	0.124	79.4	16.5	986
EfficientViM-M4	-	256 ²	300	SSD	8,170	$\times 1.00$	0.122	79.4	19.6	1111
MobileViTV2 1.25 [39]	Arxiv 2022	256 ²	300	Att.	2,409	$\times 0.24$	0.415	79.6	7.5	2857
EfficientViM-M4	-	256 ²	450	SSD	8,170	$\times 1.00$	0.122	79.6	19.6	1111

Table 3. **Comparison of efficient networks on ImageNet-1K [9] classification.** Results are sorted by accuracy. We also denote the relative throughput Thr_{rel} of each method compared to EfficientViM in each split.

Method	Thr.	Thr _{rel}	Top-1	Params	FLOPs
EfficientViM-M2	17,005	$\times 2.08$	75.8	13.9M	355M
ViM-T [77]	1,612	$\times 0.20$	76.1	7.1M	1500M
LocalViM-T [77]	593	$\times 0.07$	76.2	8.0M	1500M
EfficientVMamba-T [37]	2,763	$\times 0.34$	76.5	6.0M	800M
MSVMamba-N [50]	2,060	$\times 0.25$	77.3	6.9M	864M
EfficientVMamba-S [37]	1,350	$\times 0.17$	78.7	11.0M	1300M
EfficientViM-M4	8,170	$\times 1.00$	79.6	19.6M	1111M
MSVMamba-M [50]	1,527	$\times 0.19$	79.8	11.9M	1507M

Table 4. **Comparison of EfficientViM with vision Mambas.** Thr_{rel} is relative throughput compared to EfficientViM-M4.

Comparison with vision Mambas. In Table 4, we compare our EfficientViM with the recent vision Mambas including ViM [77], LocalViM [21], EfficientVMamba [42], and MSVMamba [50]. EfficientViM brings promising speed improvements over the prior works. EfficientViM-M2 is almost $10\times$ and $29\times$ faster than ViM-T and LocalViM-T with comparable accuracy. EfficientViM-M4 shows about $3\sim 14\times$ higher throughput than other methods even with competitive performances. Notably, it outperforms MSVMamba-N and EfficientVMamba-S by 3.1% and 2.3%, achieving about $4\times$ and $6\times$ higher throughput, respectively. This reveals that EfficientViM is a highly efficient architecture among mamba-based vision models.

Method	Size	Thr.	Thr _{rel}	Top-1	Params	FLOPs
SHViT-S1 [72]	224 ²	19,868	×0.96	74.0	6.3M	241M
EfficientViM-M1	224 ²	20,731	×1.00	74.6	6.7M	239M
EfficientFormerV2-S0 [30]	224 ²	1,350	×0.08	75.7	3.5M	407M
SwiftFormer-XS [47]	224 ²	6,102	×0.36	75.7	3.5M	605M
SHViT-S2 [72]	224 ²	15,672	×0.94	76.2	11.4M	366M
FastViT-T8 [58]	256 ²	4,365	×0.26	76.7	3.6M	705M
EfficientViM-M2	224 ²	17,005	×1.00	76.7	13.9M	355M
EfficientMod-XXS [37]	224 ²	7,022	×0.59	77.1	4.7M	583M
SHViT-S3 [72]	224 ²	11,873	×0.99	78.3	14.2M	601M
SwiftFormer-S [47]	224 ²	4,675	×0.39	78.5	6.1M	988M
EfficientFormerV2-S1 [30]	224 ²	1,248	×0.10	79.0	6.1M	668M
EfficientViM-M3	224 ²	11,952	×1.00	79.1	16.6M	656M
EfficientMod-XS [37]	224 ²	5,321	×0.65	79.4	6.6M	778M
SHViT-S4 [72]	256 ²	8,024	×0.98	80.2	16.5M	986M
FastViT-T12 [58]	256 ²	2,741	×0.34	80.3	6.8M	1419M
EfficientViM-M4	256 ²	8,170	×1.00	80.7	19.6M	1111M

Table 5. Comparison of efficient networks after training with distillation objective in [55]. Thr_{rel} is relative throughput compared to EfficientViM in each split.

Head: Mask R-CNN [16]							
Method	Lat. (ms)	AP ^b	AP ^b ₅₀	AP ^b ₇₅	AP ^m	AP ^m ₅₀	AP ^m ₇₅
EfficientNet-B0 [52]	0.95	31.9	51.0	34.5	29.4	47.9	31.2
PoolFormer-S1 [71]	1.49	37.3	59.0	40.1	34.6	55.8	36.9
FastViT-SA12 [58]	1.63	38.9	60.5	42.2	35.9	57.6	38.1
SHViT-S4 [72]	0.52	39.0	61.2	41.9	35.9	57.9	37.9
EfficientViM-M4	0.45	39.3	60.2	42.5	35.8	57.1	37.4
Head: RetinaNet [45]							
Method	Lat. (ms)	AP	AP ₅₀	AP ₇₅	AP _s	AP _m	AP _l
PVTv2-B0 [63]	0.87	37.2	57.2	39.5	23.1	40.4	49.7
MobileFormer-508M [3]	1.09	38.0	58.3	40.3	22.9	41.2	49.7
EdgeViT-XXS [41]	0.94	38.7	59.0	41.0	22.4	42.0	51.6
SHViT-S4 [72]	0.52	38.8	59.8	41.1	22.0	42.4	52.7
EfficientViM-M4	0.45	38.8	59.6	41.1	22.1	42.4	52.8

Table 6. Instance segmentation and object detection and results on COCO-2017 [31].

Training with distillation. We compare EfficientViM with the prior works [30, 47, 58, 72] trained with distillation objectives in DeiT [55]. We train the model for 300 epochs, using RegNetY-160 [43] as the teacher model. Table 5 shows that distillation is effective for EfficientViM. Compared to FastViT-T8&T12 [58], EfficientViM-M2&M4 delivers more than $3\times$ higher throughput along with comparable or even better performance. Further, EfficientViM outperforms SHViT [72] up to 0.8% running at a higher speed. With distillation, EfficientViM still outperforms all other models in speed-accuracy trade-offs and further establishes a promising Pareto front as shown in Figure 1.

4.2. Dense Predictions

Object detection and instance segmentation. We validate the effectiveness of EfficientViM on object detection and instance segmentation using the COCO-2017 [31] dataset. For training the models, we follow the settings of previous

Metric	PVTv2-B0	FastViT-SA12	EdgeViT-XXS	EfficientViM-M4
Latency (ms)	0.87	1.63	0.94	0.45
mIoU(%)	37.2	38.0	39.7	41.3

Table 7. Semantic segmentation with SemanticFPN [25] on ADE20K [76]

Method	Memory	Thr.	Thr _{rel}	Top-1	Params
EfficientViT-M4 [33]	870M	15,807	×0.93	74.3	8.8M
EMO-2M [73]	2656M	4,990	×0.29	75.1	2.3M
MobileNetV3-L 1.0 [18]	2643M	9,493	×0.56	75.2	5.4M
MobileFormer-151M [3]	1800M	8,890	×0.52	75.2	7.6M
SHViT-S2 [72]	879M	15,899	×0.94	75.2	11.4M
FastViT-T8 [72]	2811M	4,365	×0.26	75.6	3.6M
EfficientViM-M2	969M	17,005	×1.00	75.8	13.9M

Table 8. Comparison on peak memory usage during inference. Thr_{rel} is relative throughput compared to EfficientViM-M2.

works [33, 41, 58, 72], where Mask R-CNN [16] and RetinaNet [45] are used for instance segmentation and object detection. After training the model for 12 epochs (1x schedule) with a batch size of 16, we report the performances and backbone latencies with the resolution of 512^2 , following [72]. Table 6 demonstrates that EfficientViM achieves competitive performances while maintaining a faster speed in dense prediction tasks. Specifically, in instance segmentation with Mask R-CNN, EfficientViM-M4 surpasses SHViT-S4 with the 0.3% improvements in AP^b while reducing latency by 0.7ms. Similarly, in object detection with RetinaNet, EfficientViM-M4 achieves the best average precision of 38.8% with the lowest latency.

Semantic segmentation. We also demonstrate the extensibility of EfficientViM on semantic segmentation using ADE20K [76] benchmark. Following previous works [41, 58], we replace the backbone of SemanticFPN [25] with EfficientViM-M4. The model is finetuned for 40K iterations with the batch size of 32 and the initial learning rate of 2×10^{-4} decayed using a polynomial scheduler with a power of 0.9. We report the mIoU and latency of the backbone with 512^2 resolutions in Table 7. We observe that EfficientViM-M4 largely surpasses all previous efficient backbones in semantic segmentation, which demonstrates the efficacy of EfficientViM in dense predictions along with the results of object detection and instance segmentation. It is worth noting that EfficientViM-M4 brings both 1.6% mIoU gains and about $2\times$ speedup compared to the second-best model EdgeViT-XXS.

4.3. Analysis and Ablation studies

Memory Efficiency. Our models have a relatively large number of parameters compared to prior works. Yet, the memory usage of the model in the device is determined by the memory I/O during inference rather than just the number of parameters alone. We here analyze the peak memory usage of the networks and provide the results in Table 8.

Method	Thr.	Top-1	Params	FLOPs
(A) EfficientViM-M2 (Base)	17,005	75.4	13.9M	355M
(B) EfficientViM-M3	11,952	77.5	16.6M	656M
<i>a. Token Mixers (Base: HSM-SSD)</i>				
(C) ($\rightarrow$) NC-SSD [49]	9,786	76.2	13.0M	382M
(D) ($\rightarrow$) Self-Attention [60]	13,038	76.1	13.6M	416M
<i>b. Head designs (Base: Single-head with $\mathbf{A} \in \mathbb{R}^{L \times N}$)</i>				
(E) ($\rightarrow$) Multi-head	15,703	75.4	13.9M	352M
<i>c. Multi-stage fusion</i>				
(F) ($\rightarrow$) None	17,317	75.1	13.0M	354M

Table 9. **Ablation studies on EfficientViM.** All ablation studies were conducted with EfficientViM-M2 denoted as (Base).

Despite the highest parameter counts, EfficientViM shows a competitive memory efficiency while achieving the best throughput. Notably, we observe that EfficientViM requires only about 1/3 of the peak memory usage of the models having lower parameters, *e.g.*, EMO [73], MobileNetV3 [18], and FastViT [58]. Furthermore, the models with a relatively large number of parameters (*e.g.*, EfficientViT, SHViT, and EfficientViM) demonstrate high throughput and low memory consumption, highlighting that the number of parameters is not a critical factor for memory and time efficiency. Overall, our EfficientViM achieves the best speed and performance maintaining memory efficiency.

Ablation studies. We conduct ablation studies with EfficientViM-M2 after training 300 epochs and provide the results in Table 9. First, we compare HSM-SSD with other global token mixers, by replacing them with other methods including NC-SSD [49] and self-attention (SA) [60]. Considering that EfficientViM-M3 with HSM-SSD shows 77.5% with a throughput of 11,952 (im/s), NC-SSD and SA show a poorer speed-accuracy trade-off than HSM-SSD. Regarding head choices, we observe that our single-head design brings significant speed-up (15703 $\rightarrow$ 17005 (im/s)) without performance degradation. Lastly, multi-stage fusion with hidden states (75.4%) surpasses the accuracy of EfficientViM without fusion (75.1%) under similar throughput. Refer to the supplement for more ablation studies.

5. Related Works

Efficient vision backbones. Earlier works [5, 18, 19, 22, 36, 46, 52, 59, 74] have studied improving the trade-off between accuracy and computational efficiency in CNN architectures. To reduce the complexity of convolution, Xception [5] introduced depthwise convolutions (DWConv), which have become major techniques used across modern efficient models. For instance, the pioneering work MobileNet [19] constructed lightweight architectures based on DWConv. ShuffleNet [74] and GhostNet [15] also have explored additional techniques to enhance CNN by shuffling

the channel and generating more feature maps.

Following ViTs [11], several works [3, 33, 38, 39, 41, 58, 72, 73] built efficient vision backbones with attention [60]. Some works sparsified the attention [7, 10, 35] to reduce query and key, while another line of works [6, 26, 62, 66] have approximated attention itself with reduced cost. More recently, EfficientFormer [29], EfficientViT [33], and SHViT [72] have proposed hardware-friendly ViT architectures for real-world applications, focusing on actual speed in practical use, rather than FLOPs.

Vision Mambas. State space model (SSM) [8, 12–14, 51] has become a popular global token mixer with its favorable linear cost. Especially, Mamba [13] has introduced a selective scan mechanism on SSM (S6) to enable time-variant selection. Following Mambas [8, 13], several works [21, 34, 37, 48, 54, 69, 77] have proposed SSM-based Vision backbones. ViM [77] applied bi-directional SSM to flattened patches considering the causal nature in Mamba. Similarly, VMamba [34], PlainMamba [69], and LocalMamba [21] have been proposed with the multi-path scanning mechanism. GroupMamba [48], and EfficientV-Mamba [42] further enhance the efficiency by splitting the channels and sharing the learnable parameters for each path. Recent works, VSSD [49] and Linfusion [32] introduced non-causal SSD to overcome causal properties in Mamba.

6. Conclusion

We propose a novel mamba-based architecture, Efficient Vision Mamba (EfficientViM), built with a hidden state mixer-based SSD (HSM-SSD). With the observation that the primary bottleneck of SSD stems from the linear projections with input tokens, we rearrange the channel mixing operation within the hidden states which serve as reduced latent representations. We also introduce multi-stage hidden state fusion that integrates both low-level and high-level features. To enhance practical runtime efficiency, we adopt a single-head design for HSM-SSD to minimize the memory-bound operations, which are typically overlooked when only considering FLOPs. Our comprehensive experiments demonstrate that EfficientViM significantly improves efficiency and performance over prior works across various tasks.

Acknowledgments. This work was supported by Korea University - KT (Korea Telecom) R&D Center, the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (NRF-2023R1A2C2005373), the Virtual Engineering Platform Project (Grant No. P0022336), funded by the Ministry of Trade, Industry & Energy (MoTIE, South Korea), and the Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (No. RS-2024-00457882, AI Research Hub Project).

References

- [1] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster. *ICLR*, 2022. 1
- [2] Yinpeng Chen, Xiyang Dai, Mengchen Liu, Dongdong Chen, Lu Yuan, and Zicheng Liu. Dynamic relu. In *ECCV*, 2020. 5
- [3] Yinpeng Chen, Xiyang Dai, Dongdong Chen, Mengchen Liu, Xiaoyi Dong, Lu Yuan, and Zicheng Liu. Mobileformer: Bridging mobilenet and transformer. In *CVPR*, 2022. 1, 6, 7, 8
- [4] Joonmyung Choi, Sanghyeok Lee, Jaewon Chu, Minhyuk Choi, and Hyunwoo J Kim. vid-tldr: Training free token merging for light-weight video transformer. In *CVPR*, 2024. 1
- [5] François Chollet. Xception: Deep learning with depthwise separable convolutions. In *CVPR*, 2017. 1, 8
- [6] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. *ICLR*, 2021. 1, 8
- [7] Xiangxiang Chu, Zhi Tian, Yuqing Wang, Bo Zhang, Haibing Ren, Xiaolin Wei, Huaxia Xia, and Chunhua Shen. Twins: Revisiting the design of spatial attention in vision transformers. *NeurIPS*, 2021. 1, 8
- [8] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024. 1, 2, 3, 4, 8
- [9] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009. 1, 5, 6
- [10] Xiaoyi Dong, Jianmin Bao, Dongdong Chen, Weiming Zhang, Nenghai Yu, Lu Yuan, Dong Chen, and Baining Guo. Cswin transformer: A general vision transformer backbone with cross-shaped windows. In *CVPR*, 2022. 8
- [11] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*, 2021. 1, 8
- [12] Daniel Y Fu, Tri Dao, Khaled K Saab, Armin W Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models. *ICLR*, 2023. 1, 8
- [13] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*, 2023. 2, 8
- [14] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *ICLR*, 2022. 1, 8
- [15] Kai Han, Yunhe Wang, Qi Tian, Jianyuan Guo, Chunjing Xu, and Chang Xu. Ghostnet: More features from cheap operations. In *CVPR*, 2020. 1, 8
- [16] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *ICCV*, 2017. 7
- [17] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv:1606.08415*, 2016. 5
- [18] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. Searching for mobilenetv3. In *ICCV*, 2019. 1, 2, 5, 6, 7, 8
- [19] Andrew G Howard. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*, 2017. 1, 8
- [20] Jie Hu, Linyan Huang, Tianhe Ren, Shengchuan Zhang, Rongrong Ji, and Liujuan Cao. You only segment once: Towards real-time panoptic segmentation. In *CVPR*, 2023. 1
- [21] Tao Huang, Xiaohuan Pei, Shan You, Fei Wang, Chen Qian, and Chang Xu. Localmamba: Visual state space model with windowed selective scan. *arXiv:2403.09338*, 2024. 2, 6, 8
- [22] Forrest N Iandola. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and 0.5 mb model size. *arXiv:1602.07360*, 2016. 1, 8
- [23] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. Perceiver: General perception with iterative attention. In *International conference on machine learning*, 2021. 4
- [24] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *ICML*, pages 5156–5165. PMLR, 2020. 3
- [25] Alexander Kirillov, Ross Girshick, Kaiming He, and Piotr Dollár. Panoptic feature pyramid networks. In *CVPR*, 2019. 7
- [26] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *ICLR*, 2020. 1, 8
- [27] Sanghyeok Lee, Joonmyung Choi, and Hyunwoo J Kim. Multi-criteria token fusion with one-step-ahead attention for efficient vision transformers. In *CVPR*, 2024. 1
- [28] Quanquan Li, Shengying Jin, and Junjie Yan. Mimicking very efficient network for object detection. In *CVPR*, 2017. 1
- [29] Yanyu Li, Geng Yuan, Yang Wen, Ju Hu, Georgios Evangelidis, Sergey Tulyakov, Yanzhi Wang, and Jian Ren. Efficientformer: Vision transformers at mobilenet speed. *NeurIPS*, 2022. 8
- [30] Yanyu Li, Ju Hu, Yang Wen, Georgios Evangelidis, Kamyar Salahi, Yanzhi Wang, Sergey Tulyakov, and Jian Ren. Rethinking vision transformers for mobilenet size and speed. In *ICCV*, 2023. 6, 7
- [31] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *ECCV*, 2014. 7
- [32] Songhua Liu, Weihao Yu, Zhenxiong Tan, and Xinchao Wang. Linfusion: 1 gpu, 1 minute, 16k image. *arXiv:2409.02097*, 2024. 2, 8
- [33] Xinyu Liu, Houwen Peng, Ningxin Zheng, Yuqing Yang, Han Hu, and Yixuan Yuan. Efficientvit: Memory efficient vision transformer with cascaded group attention. In *CVPR*, 2023. 1, 5, 6, 7, 8
- [34] Yue Liu, Yunjie Tian, Yuzhong Zhao, Hongtian Yu, Lingxi Xie, Yaowei Wang, Qixiang Ye, and Yunfan Liu. Vmamba: Visual state space model. *NeurIPS*, 2024. 2, 8

- [35] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021. 1, 8
- [36] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. Shufflenet v2: Practical guidelines for efficient cnn architecture design. In *ECCV*, 2018. 1, 8
- [37] Xu Ma, Xiyang Dai, Jianwei Yang, Bin Xiao, Yinpeng Chen, Yun Fu, and Lu Yuan. Efficient modulation for vision networks. *ICLR*, 2024. 6, 7, 8
- [38] Sachin Mehta and Mohammad Rastegari. Mobilevit: light-weight, general-purpose, and mobile-friendly vision transformer. *ICLR*, 2022. 1, 8
- [39] Sachin Mehta and Mohammad Rastegari. Separable self-attention for mobile vision transformers. *arXiv:2206.02680*, 2022. 1, 5, 6, 8
- [40] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *ICML*, 2010. 5
- [41] Junting Pan, Adrian Bulat, Fuwen Tan, Xiatian Zhu, Lukasz Dudziak, Hongsheng Li, Georgios Tzimiropoulos, and Brais Martinez. Edgevits: Competing light-weight cnns on mobile devices with vision transformers. In *ECCV*, 2022. 1, 6, 7, 8
- [42] Xiaohuan Pei, Tao Huang, and Chang Xu. Efficientv-mamba: Atrous selective scan for light weight visual mamba. *arXiv:2403.09977*, 2024. 6, 8
- [43] Ilija Radosavovic, Raj Prateek Kosaraju, Ross Girshick, Kaiming He, and Piotr Dollár. Designing network design spaces. In *CVPR*, 2020. 7
- [44] J Redmon. You only look once: Unified, real-time object detection. In *CVPR*, 2016. 1
- [45] T-YLPG Ross and GKHP Dollár. Focal loss for dense object detection. In *CVPR*, 2017. 7
- [46] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *CVPR*, 2018. 1, 5, 8
- [47] Abdelrahman Shaker, Muhammad Maaz, Hanoona Rasheed, Salman Khan, Ming-Hsuan Yang, and Fahad Shahbaz Khan. Swiftformer: Efficient additive attention for transformer-based real-time mobile vision applications. In *ICCV*, 2023. 7
- [48] Abdelrahman Shaker, Syed Talal Wasim, Salman Khan, Juergen Gall, and Fahad Shahbaz Khan. Groupmamba: Parameter-efficient and accurate group visual state space model. *arXiv:2407.13772*, 2024. 8
- [49] Yuheng Shi, Mingjing Dong, Mingjia Li, and Chang Xu. Vssd: Vision mamba with non-casual state space duality. *arXiv:2407.18559*, 2024. 2, 3, 4, 8
- [50] Yuheng Shi, Mingjing Dong, and Chang Xu. Multi-scale vmamba: Hierarchy in hierarchy visual state space model. *NeurIPS*, 2024. 2, 6
- [51] Jimmy TH Smith, Andrew Warrington, and Scott W Linderman. Simplified state space layers for sequence modeling. *ICLR*, 2023. 1, 8
- [52] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *ICML*, 2019. 1, 7, 8
- [53] Mingxing Tan, Ruoming Pang, and Quoc V Le. Efficientdet: Scalable and efficient object detection. In *CVPR*, 2020. 1
- [54] Lv Tang, HaoKe Xiao, Peng-Tao Jiang, Hao Zhang, Jinwei Chen, and Bo Li. Scalable visual state space model with fractal scanning. *arXiv:2405.14480*, 2024. 8
- [55] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *ICML*, 2021. 1, 7
- [56] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers. In *ICCV*, 2021. 5
- [57] Hugo Touvron, Matthieu Cord, and Hervé Jégou. Deit iii: Revenge of the vit. In *ECCV*, 2022. 5
- [58] Pavan Kumar Anasosalu Vasu, James Gabriel, Jeff Zhu, Oncel Tuzel, and Anurag Ranjan. Fastvit: A fast hybrid vision transformer using structural reparameterization. In *ICCV*, 2023. 1, 5, 6, 7, 8
- [59] Pavan Kumar Anasosalu Vasu, James Gabriel, Jeff Zhu, Oncel Tuzel, and Anurag Ranjan. Mobileone: An improved one millisecond mobile backbone. In *CVPR*, 2023. 1, 5, 6, 8
- [60] A Vaswani. Attention is all you need. *NeurIPS*, 2017. 3, 4, 8
- [61] Chengcheng Wang, Wei He, Ying Nie, Jianyuan Guo, Chuanjian Liu, Yunhe Wang, and Kai Han. Gold-yolo: Efficient object detector via gather-and-distribute mechanism. *NeurIPS*, 2024. 1
- [62] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv:2006.04768*, 2020. 1, 8
- [63] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pvt2: Improved baselines with pyramid vision transformer, 2022. 7
- [64] Sanghyun Woo, Shoubhik Debnath, Ronghang Hu, Xinlei Chen, Zhuang Liu, In So Kweon, and Saining Xie. Convnex v2: Co-designing and scaling convnets with masked autoencoders. In *CVPR*, 2023. 6
- [65] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M Alvarez, and Ping Luo. Segformer: Simple and efficient design for semantic segmentation with transformers. *NeurIPS*, 2021. 1
- [66] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nyström-based algorithm for approximating self-attention. In *AAAI*, 2021. 1, 8
- [67] Yunyang Xiong, Bala Varadarajan, Lemeng Wu, Xiaoyu Xiang, Fanyi Xiao, Chenchen Zhu, Xiaoliang Dai, Dilin Wang, Fei Sun, Forrest Iandola, Raghuraman Krishnamoorthi, and Vikas Chandra. EfficientSAM: Leveraged masked image pre-training for efficient segment anything. In *CVPR*, 2024. 1
- [68] Jiacong Xu, Zixiang Xiong, and Shankar P Bhattacharyya. Pidnet: A real-time semantic segmentation network inspired by pid controllers. In *CVPR*, 2023. 1
- [69] Chenhongyi Yang, Zehui Chen, Miguel Espinosa, Linus Ericsson, Zhenyu Wang, Jiaming Liu, and Elliot J Crowley. Plainmamba: Improving non-hierarchical mamba in visual recognition. *BMVC*, 2024. 2, 8

- [70] Hongxu Yin, Arash Vahdat, Jose M Alvarez, Arun Mallya, Jan Kautz, and Pavlo Molchanov. A-vit: Adaptive tokens for efficient vision transformer. In *CVPR*, 2022. [1](#)
- [71] Weihao Yu, Mi Luo, Pan Zhou, Chenyang Si, Yichen Zhou, Xinchao Wang, Jiashi Feng, and Shuicheng Yan. Metaformer is actually what you need for vision. In *CVPR*, 2022. [7](#)
- [72] Seokju Yun and Youngmin Ro. Shvit: Single-head vision transformer with memory efficient macro design. In *CVPR*, 2024. [1](#), [2](#), [4](#), [5](#), [6](#), [7](#), [8](#)
- [73] Jiangning Zhang, Xiangtai Li, Jian Li, Liang Liu, Zhucun Xue, Boshen Zhang, Zhengkai Jiang, Tianxin Huang, Yabiao Wang, and Chengjie Wang. Rethinking mobile block for efficient attention-based models. In *ICCV*, 2023. [1](#), [6](#), [7](#), [8](#)
- [74] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *CVPR*, 2018. [1](#), [8](#)
- [75] Hengshuang Zhao, Xiaojuan Qi, Xiaoyong Shen, Jianping Shi, and Jiaya Jia. Icnet for real-time semantic segmentation on high-resolution images. In *ECCV*, 2018. [1](#)
- [76] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ade20k dataset. In *CVPR*, 2017. [7](#)
- [77] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. Vision mamba: Efficient visual representation learning with bidirectional state space model. *ICML*, 2024. [2](#), [5](#), [6](#), [8](#)